

实验二

年级：_____2022_____专业：_____计算机科学与技术_____

姓名：_____姚知言_____学号：_____2211290_____

1 实验内容

(1) 使用数组实现栈的出栈、入栈、获取栈顶元素、判断空栈四个基本方法，并在此基础上实现一个能够得到栈最大元素的 `get_max()` 方法。

(2) 输入一组数字存入 (1) 所创建的栈中，首先获取这组数字中的最大值并输出，然后删除该组数字中两个相邻且相同的数字，反复操作直到无法删除重复项，并返回最终的数字序列，保证答案唯一。

输入：11, 6, 1, 2, 2, 1, 6, 2

输出：11 11, 2

(3) 使用该 (1) 中的栈进行表达式运算。表达式包含 “+”、“-”、“*”、“/”、“(”、“)” 和数字的字符串，输出该表达式运算结果。

输入：((1+2)*3-5)/2

输出： 2

实验要求：不能使用调用外部库实现栈。

2 设计思想

该程序的设计思想如下：

根据题目要求，定义 `stack` 类，使用数组实现栈的操作，并完成构造函数和析构函数的定义。

类内定义 `pop` 函数和 `push` 函数实现出栈，入栈操作。并且在栈满和栈空的

时候实现异常处理。

类内定义 findtop 函数和 isempty 函数，完成获取栈顶元素和判断空栈的操作。并通过遍历栈内元素，完成 get_max 方法。

类内定义 ex_pop 函数，完成删除栈内从栈底开始计数的第 start 个数开始向栈顶方向的共计 pace 个数。

类内定义 print 函数，实现（2）对处理后栈的输出（栈底至栈顶），也为后续 debug 减轻难度。

类内定义 samefinder 函数，从栈底开始寻找回文序列（由于数组从之间删除某个元素操作时间复杂度较大，根据本题描述，回文序列可以一并被删除，从而有效降低大数据的时间复杂度）。并定义 samebreaker 函数，循环调用 samefinder 函数和 ex_pop 函数，完成（2）操作。

调用 oper1 函数，封装（2）中要求内容。

对于（3）：

定义 call 函数，完成表达式运算需要出栈时的出栈，数值计算及入栈操作。并在计算除法时，增加除以 0 的异常处理。

定义 calculator 函数，封装整个表达式运算操作，并能够实现输入数据为多位数及负数操作，在修改后增加了计算小数的操作（蓝色部分），同时对于异常数据（缺少，冗余运算符，运算符不匹配，输入了不符合要求的字符）进行异常处理。

3 程序效果

程序的运行效果，输入及输出的相关要求和具体执行结果如下所示：

3.1 输入

a. 题目数据组

（1）in: 11, 6, 1, 2, 2, 1, 6, 2

（2）in: ((1+2)*3-5)/2

b. 加强数据组 1

（1）in: 1, 1, 4, 5, 1, 4, 8, 9, 9, 10, 11, 11, 10, 8, 6

（2）in: (-4+385+(-6)*2)/3

c. 加强数据组 2

(1) in: -1, -2, -3, -4, -5, -6, -6, -5, -4, -3

(2) in: $333/(5/5-1)$

3.2 输出

a. 题目数据组

(1) out1: 11

(1) out2: 11, 2

(2) out: 2

b. 加强数据组 1

(1) out1: 11

(1) out2: 4, 5, 1, 4, 6

(2) out: 123

c. 加强数据组 2

(1) out1: -1

(1) out2: -1, -2

(2) out: terminate called after throwing an instance of
'std::invalid_argument'

what(): Divided by 0!

4 核心代码

```
#include <iostream>
```

```
#include <cstring>
```

```
using namespace std;
```

```
template<class T>
```

```
class stack{
```

```
public:
```

```
    stack(int maxn=1000000);
```

```
    ~stack();
```

```

    bool isempty();
    T findtop();
    void push(T x);
    T pop();
    void ex_pop(int& start, int& pace);
    T get_max();
    void print();
    void samefinder(int& start, int& pace);
    void samebreaker();
private:
    int top;
    int maxtop;
    T* mstack;
};

template<class T>
stack<T>::stack(int maxn) {
    top=0;
    maxtop=maxn;
    mstack=new T[maxn];
}

template<class T>
stack<T>::~~stack() {
    delete[] mstack;
}

template<class T>
bool stack<T>::isempty() {
    return top==0;
}

template<class T>

```

```

T stack<T>::findtop() {
    return mstack[top-1];
}

template<class T>
void stack<T>::push(T x) {
    if(top==maxtop) {
        throw out_of_range("Out of bounds!\n");
    }
    mstack[top++]=x;
}

template<class T>
T stack<T>::pop() {
    if(!top) {
        throw out_of_range("Out of bounds!\n");
    }
    T a=mstack[--top];
    return a;
}

template<class T>
void stack<T>::ex_pop(int& start,int& pace) {
    for(int a=start;a+pace<top;a++) {
        mstack[a]=mstack[a+pace];
    }
    top-=pace;
}

template<class T>
T stack<T>::get_max() {
    if(!top) {
        throw out_of_range("No elements!\n");
    }
}

```

```

    }

    T maxn=mstack[0];
    for(int p=1;p<top;p++)
        maxn=max(maxn,mstack[p]);
    return maxn;
}

template<class T>
void stack<T>::print() {
    if(isempty())return;
    cout<<mstack[0];
    for(int p=1;p<top;p++) {
        cout<<' ' <<mstack[p];
    }
    cout<<' \n' ;
}

template<class T>
void stack<T>::samefinder(int& start,int& pace) {
    for(int p=0;p<top-1;p++) {
        if(mstack[p]==mstack[p+1]) {
            start=p;
            pace=2;
            break;
        }
    }
    if(pace==0)return;
    int a=start,b=a+1;
    while(++b<top && --a>=0) {
        if(mstack[a]!=mstack[b])
            return;
    }
}

```

```

        else{
            start--;
            pace+=2;
        }
    }
}

template<class T>
void stack<T>::samebreaker() {
    while(!isempty()) {
        int start, pace=0;
        samefinder(start, pace);
        if(pace==0)
            break;
        else{
            ex_pop(start, pace);
        }
    }
}

void oper1() {
    stack<int> A;
    do {
        int x;
        cin >> x;
        A.push(x);
    } while (getchar() == ',');
    cout << A.get_max() << '\n';
    A.samebreaker();
    A.print();
}

```

```

void call(stack<double>& B, stack<char>& C) {
    double c2=B. pop(), c1=B. pop();
    char m=C. pop();
    switch(m) {
        case '+':
            c1 += c2;
            B. push(c1);
            break;
        case '-':
            c1 -= c2;
            B. push(c1);
            break;
        case '*':
            c1*=c2;
            B. push(c1);
            break;
        case '/':
            if(c2==0) throw invalid_argument("Divided by 0!");
            c1/=c2;
            B. push(c1);
            break;
    }
}

void calculator() {
    stack<double>B;
    stack<char>C;

    char          s[1000000];double          s1=0;bool
    getint=false, gettop=false, neg=false, getdot=true;double position;

    cin>>s;

```

```

for(int p=0;p<strlen(s);p++){
    if((s[p]>='0'&& s[p]<='9') || s[p]=='.') {
        if(s[p]=='.') {
            if(getdot) {
                getdot = false;
                position=0.1;
            }
            else
                throw invalid_argument("Invalid input!");
        }
        else{
            if(getdot){
                s1=s1*10+s[p]-'0';
                getint=true;
            }
            else{
                s1+=position*(s[p]-'0');
                position*=0.1;
                getint=true;
            }
        }
    }
    else {
        if (getint) {
            if(neg)
                s1*=(-1);
            neg= false;
            B.push(s1);
            s1 = 0;
        }
    }
}

```

```

    getint=false;
    getop=true;
    getdot=true;
}
if (s[p] == '+' ) {
    if(getop) {
        while (!C.isempty() && C.findtop() != '(') {
            call(B, C);
        }
        C.push(s[p]);
        getop = false;
    }
    else
        throw invalid_argument("Invalid input!");
}
else if(s[p]=='-') {
    if(getop) {
        while (!C.isempty() && C.findtop() != '(') {
            call(B, C);
        }
        C.push(s[p]);
        getop=false;
    }
    else {
        if(neg) neg=false;
        else neg=true;
    }
}
else if (s[p] == '*' || s[p] == '/') {

```

```

        if(getop) {
            while (!C.isempty() && C.findtop() != '(' &&
C.findtop() != '+' && C.findtop() != '-') {
                call(B, C);
            }
            C.push(s[p]);
            getop = false;
        }
        else
            throw invalid_argument("Invalid input!");
    }
    else if (s[p] == '(')
        C.push(s[p]);
    else if (s[p] == ')') {
        while (C.findtop() != '(') {
            call(B, C);
        }
        C.pop();
    }
    else
        throw invalid_argument("Invalid input!");
}

if(getint) {
    if (neg)
        s1 *= (-1);
    B.push(s1);
    s1 = 0;
}

```

```

while(!C.empty()){
    call(B, C);
}
double ans=B.pop();
if(!B.empty()){
    throw invalid_argument("Invalid input!");
}
cout<<ans<<' \n' ;
}
int main() {
    oper1();// (2)
    calculator();//(3)
    return 0;
}

```

5 总结

在本次实验中，主要复习了 c++ 的基本语法，对栈的了解有所加深，并了解了栈在运算中缀表达式的应用，同时复习了误差处理的相关知识。